McMaster University ENGINEERING | mohawk | W BOOTH SCHOOL OF ENGINEERING PRACTICE AND TECHNOLOGY
McMaster-Mohawk Bachelor of Technology Partnership

Laboratory 5
Servo Motor

SMRTTECH 3CC3

Cloud Computing and Internet of Things

B. Tech - SMRTTECH

OBJECTIVES

In this lab you will learn the following:

1. Learn how to build and drive **Servo Motors** using both **Potentiometers** and LabVIEW **HMIs**.

MATERIALS

1. Arduino Mega & USB Cable
2. Breadboard
3. Servo Motor
4. Potentiometer
5. 8 x Jumper Wires

PROCEDURE

Exercise 1: Create the Arduino Circuit

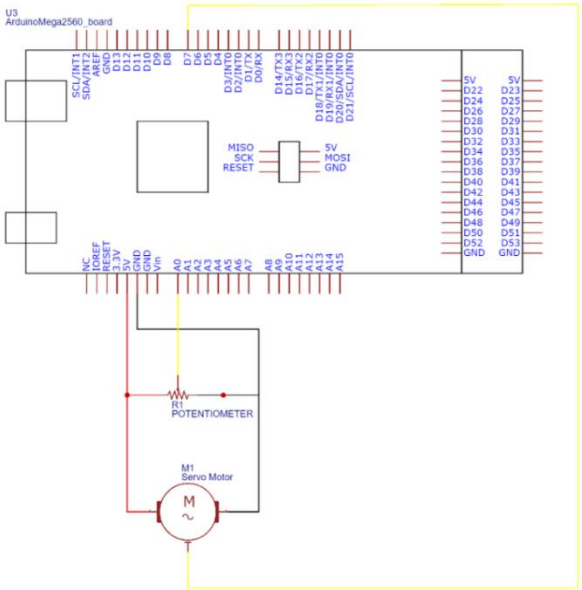
Before starting, attach the white plastic coupling by pushing it to the motor shaft.



1. Connect from the Positive (+) row on the **Breadboard** to **5V** on the **Arduino**, and from the Negative (-) row on the **Breadboard** to **Ground (GND)** on the **Arduino**.

2. Connect the Servo Motor
    a. Connect the **GND** (Brown) wire of the servo to the Negative (-) row on the **Breadboard**.
    b. Connect the **VCC** (Red) wire of the servo to the Positive (+) row on the **Breadboard**.
    c. Connect the **PWM** (Orange) wire of the servo to the Digital (7) pin on the **Arduino**.
3. Connect the Potentiometer.
    a. Connect one of the outside wires to the Negative (-) row on the Breadboard.
    b. Connect the other of the outside wires to the Positive (+) row on the Breadboard.
    c. Connect the middle wire to the Analog (0) pin on the Arduino.

4. Your circuit should now look like **Figure 1**.



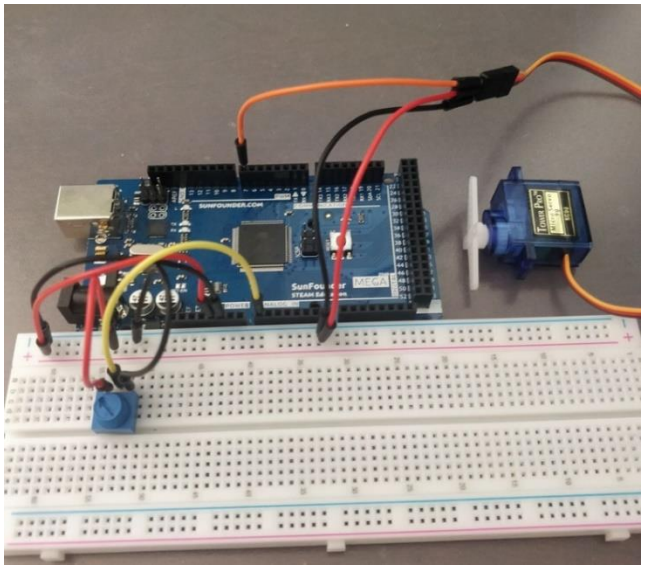


Figure 1: Arduino Circuit

End of Exercise

Exercise 2: Manual Control of the Servo using a Potentiometer

Your Goal in this exercise is to manually control the position of a servomotor using a potentiometer. This will be accomplished by doing the following:

- ***Measure the voltage of the potentiometer***
- ***Scale the voltage to an angle between 0-180°***
- ***Scale the angle to a Pulse width between 600 to 2400 uS***

1. On the **Block Diagram**, add the **Open** function from the **Functions»MakerHub»LINX** sub-menu and right-click on the Serial Port, go to **Create**, and select **Constant**, and set it to **COM3** (or the specific COM port used by your Arduino).

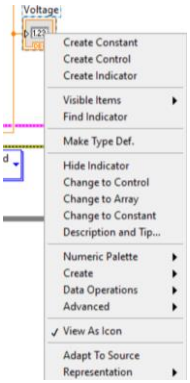
2. Under the **Functions»MakerHub»LINX»Sensors»Motion»Servo** section, add the **Open** function. Create a constant for the `Servo Channel` input and set it to `7`.

3. Place a **While Loop** from **Functions»Programming»Structures** on the **Block Diagram** and connect the `LINX Resource` and `error out` outputs to the loop with tunnels. Also, right-click on the stop-sign and create a control.

4. Inside the loop add the **Analog Read** function from the **Functions»MakerHub» LINX»Peripherals»Analog** section with `Analog Channel` input set to `0`.

5. Right click the **Voltage** pin of the **Analog Read block** and select **create indicator.**

6. Right click the voltage indicator and uncheck the **View As Icon** option.



7. Inside the loop add a **Write** (**Set Pulse Width)** function from the **Functions»MakerHub» LINX»Sensors»Motion»Servo**, with `Servo Channel` input set to `7`.

8. Set the `Pulse Width (uS)` input to `1000`.

The Pulse Width (uS) allows the user to specify the pulse width in uS to be send to the servo. A pulse width of 1500uS is typically the center point of rotation (90 degrees). Most servos can rotate between

1000 and 2000 uS but the *actual limits* will vary depending on the specific servo. For the specific servo that was used in the lab preparation, the range was between 600 to 2400 uS.

9. Add a **Close** function from **Functions»MakerHub»LINX»Sensors»Motion»Servo** and set the Servo Channel input set to `7`.

10. Add the **LINX close** function. Connect both *Arduino Resource* and *error* signals through all the blocks as shown below.

11. Your block diagram should now look like **Figure 2**.

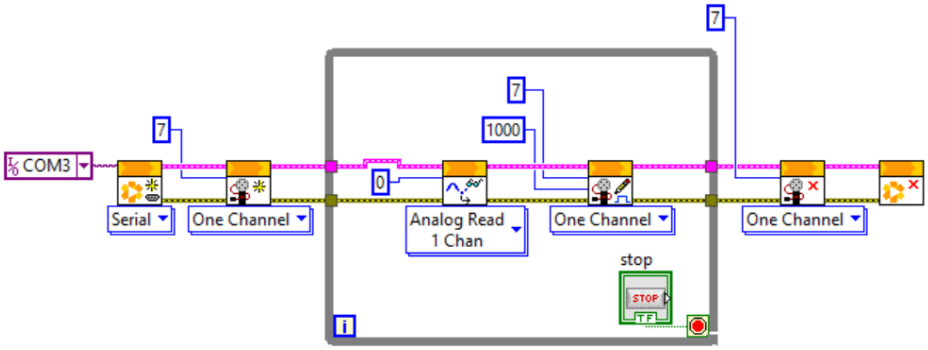


Figure 2: Arduino Interface

12. On the **Front Panel** add a **Combo Box** from the **Controls»Modern»String & Path** palette (Rename: Choice). Right-click on it and select Edit Items… and insert two items, name them Manual and Automatic.

13. Add a **Gauge** control from the **Controls»Modern»Numeric** palette (Rename: Angle). Change the scale range to `0-180`.

14. Add two **Numeric Control**s from the **Controls»Modern»Numeric** palette (Rename: Speed & NumberOfWaves).

15. Add an **OK Button** control from the **Controls»Modern»Boolean** section (Rename: GO).

16. Your panel should now look like Figure 3.

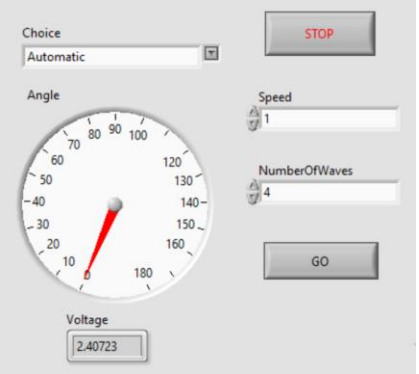
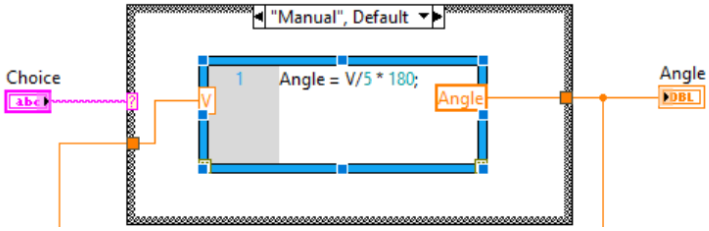
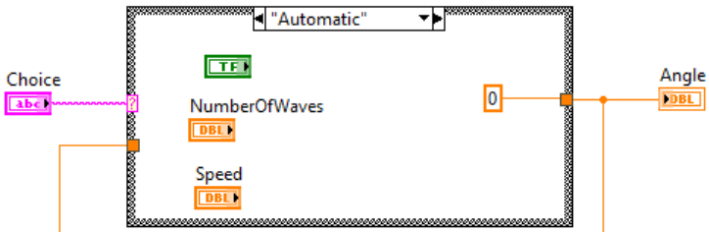


Figure 3: Front Panel

17. Go to the **Block Diagram**. Right click on every **Control and Indicator Icon** and uncheck the **View As Icon** option. Place the four input controls in the loop on the left side. Place Angle indicator on the right side. See **figure 4**.
18. Add a **Case Structure** function from the **Functions»Programming»Structures** pallet inside the loop, and connect **Choice** to the *Case Selector* input. Change the cases to "Manual", "Default" and "Automatic". See figure 4.
19. In the Manual case:
    a. Add a **Mathscript Node** from the **Functions»Mathematics»Scripts & formula** palette. A MathScript node allows to use the Matlab Scripting programming language.
    b. In the script, type `Angle=V/5*180;`
    c. Right-click on the **Mathscript Node** and click **Add Input**, rename it to V. Connect it to the voltage output from the **Analog Read Pin** function.
    d. On the **Mathscript Node** also select **Add Output**, and select **Angle**. Connect it to Angle gauge.



20. In the Automatic case:
    a. Place the **GO, Speed, NumbeOfWaves Controls** inside the Case Structure.
    b. Add a **DBL Numeric Constant** from Programming>>Numeric. Connect it to the output of the case structure as shown in the figure below.



21. Calculating the Pulse Width
    a. Below the case structure, add a **Mathscript Node** from the **Functions»Mathematics»Scripts & formula** palette.
    b. In the script, type the following code

```
URV = 2400;
LRV = 600;
PW = Angle/180*(URV-LRV) + LRV;
```

    c. Right-click on the **Mathscript Node** and click **Add Input**, rename it to Angle. Connect it to output angle from the case structure (see Figure 4).
    d. On the **Mathscript Node** also select **Add Output**, and select **PW**. Connect it to the Pulse Width input of **Set Pulse Width One Channel**. If the pulse range does not cover 180 degrees for your servo motor, you can try modifying the URV and LRV depending on your servo. Your code should now look like **Figure 4**.

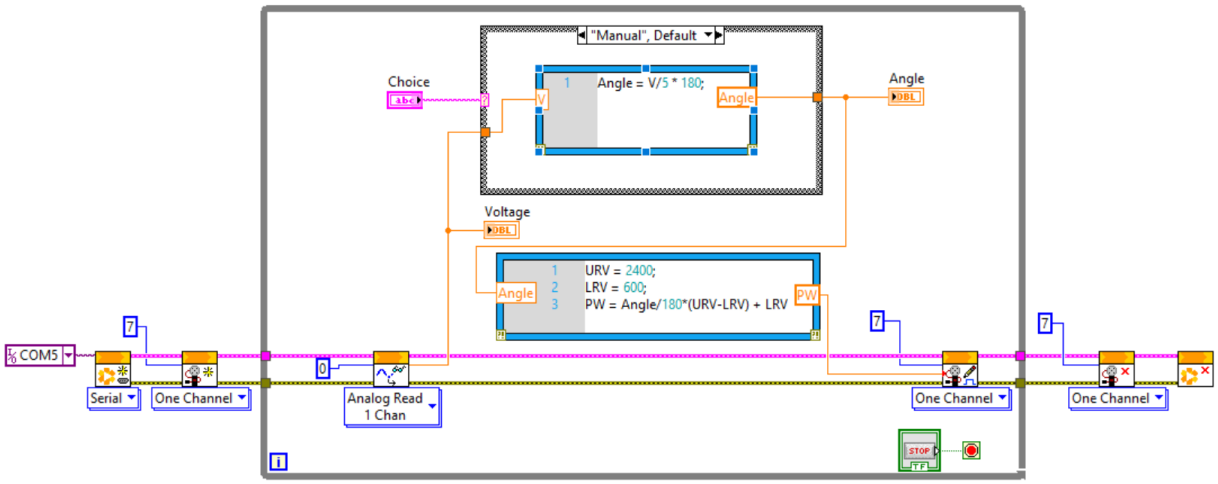


Figure 4: Manual Case

Now you can test the functionality of Manual Control of the Servo

1. ***Connect your Arduino to your computer***
2. ***Run LabVIEW program***
3. ***Turn the potentiometer With a screw driver and observe the response of the servo***

Notes:

a. (**optional**) If you want to invert the direction of rotation of the servo so that it turns in-sync with the potentiometer, you will need to swap the connection of the two potentiometer terminals that are connected to the ground and +5V.

End of Exercise

Exercise 3: Creating a SubVI for Automatic Control of The Servo

Your Goal in exercises 3 and 4 is to automatically control the position of a servomotor. Imagine that the servo is used to rotate a robot arm that greets visitors by waving to them. Your LabVIEW application should allow the user to do the following:

- ***Enter the speed of servo movement***
- ***Enter the number of Waves***
- ***When the user presses the GO button, the servo will move at the desired speed to perform the desired number of waves.***

1. Open a new **Blank VI**.
2. Add the following to your front panel:

    a. One **Button** (Rename: Go)
    b. Two **LEDs** (Rename: Waving and Direction) from the **Boolean** section
    c. Two **Numeric Indicators** (Rename: Wave Count and Angle)
    d. Two **Numeric Controls** (Rename: NumberOfWaves and Speed) from the **Numeric** section.

3. The front panel should look like **figure 5**.

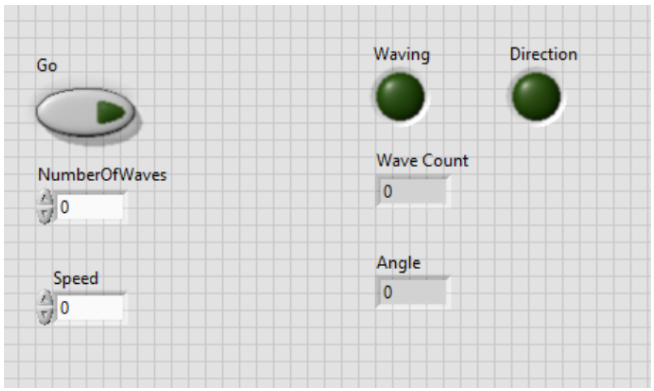


Figure 5: Front Panel

4. In the **block diagram**, add a **Mathscript Node** from the **Functions»Mathematics»Scripts & formula** palette. A MathScript node allows to use the Matlab Scripting programming language.

5. Copy and paste the following code inside the node:

```
Angle = AngleLast;
Waving = WavingLast;
Direction = DirectionLast;
WaveCount = WaveCountLast;

if Go == true
    Waving = true
    WaveCount = NumberOfWaves
end
```

```
if Waving == true

    if Direction == false
        Angle = Angle+ Speed
    else
        Angle = Angle - Speed
    end

    if Angle>=180 |  Angle<=0
        Direction = Not (Direction)
        WaveCount = WaveCount - 1
    end

    if WaveCount == 0
        Waving = false
    end

end
```

6. Right click on the left side of the node and select **add input**. Add all the inputs as shown.

7. Right click on the right side of the node and select **add output**. Add all the outputs as shown.

8. From the **function** palette, search and select "**local variable**" and place it on your block diagram. Left click the local variable and select **Angle**. Right click the variable and select change to **Read**. Connect the output of the local variable to the **AngleLast** Input of the MathScript. See figure 6.

9. Repeat for all other local variables shown in Figure 6.

10. Lastly, add a **Wait** function, found under **Functions»Programming»Timing**, and create a constant for the input set to 10.

11. You code should now look like **Figure 6**.

12. In the top-right corner of the **Front Panel**, right-click, select **Edit Icon…** and change it to something better. **E.g**. "Wave Servo".

13. In the top-right corner of the **Front Panel**, right-click, select **Patterns…** and change it to be three inputs an one output.
14. Connect the top input to the **GO** button, the middle input to the **NumberOfWaves** control, the bottom input to the **Speed** control, and connect the output to Angle.



Figure 7: SubVI

15. Save the program and go back to the Main VI.

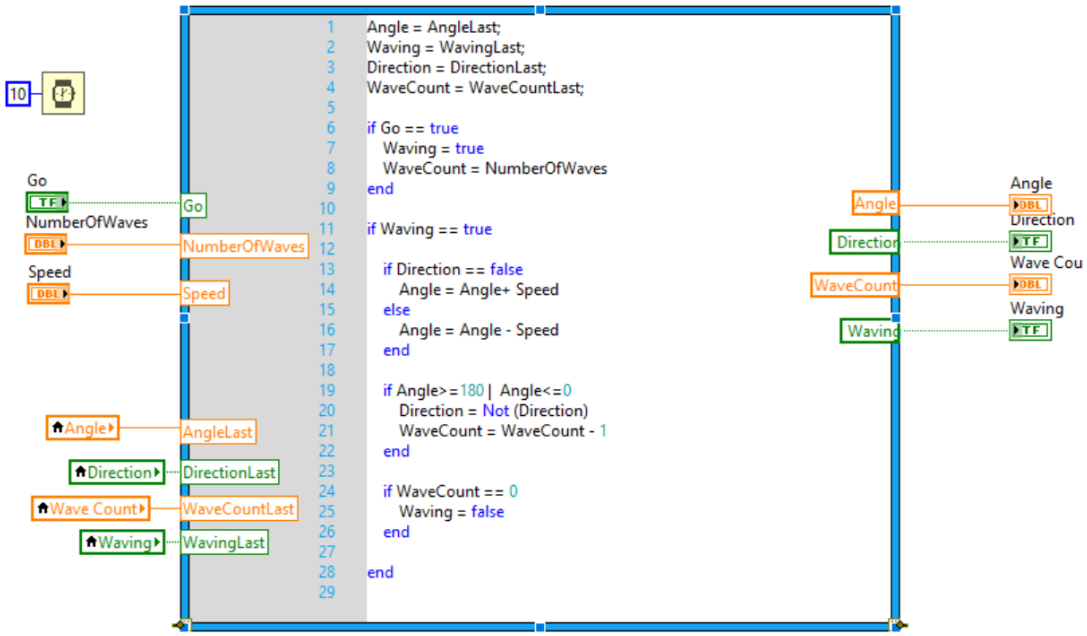


Figure 6: Block Diagram

End of Exercise

Exercise 4: Automatic Control of the Servo

16. Return to the main VI that you created in Exercise 2.
17. In the **Functions Panel**, click **Select a VI…** and find your SubVI. Place it in the Automatic case of the **Case Structure**.
18. Delete the **DBL Numeric Constant** from the Automatic case of the **Case Structure** and connect the output of the subVI to the output of the case structure as shown in Figure 8.
19. Connect your SubVI to the Go, NumberOfWaves, and Speed.
20. Your final program should look like **Figure 8**.

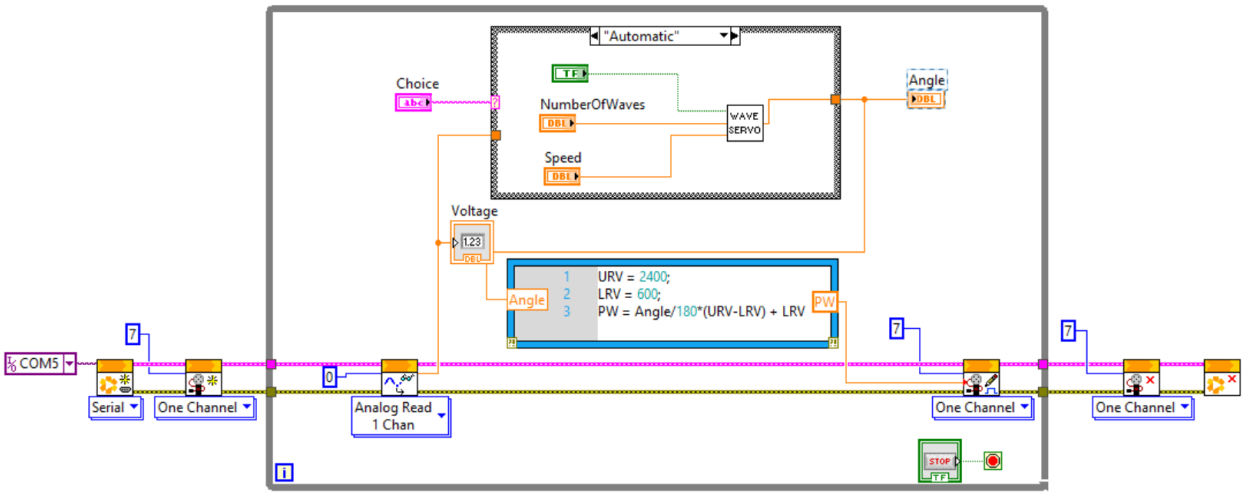


Figure 8: Final Program

21. Test your program and confirm its functionality.

End of Exercise

SUBMISSION

the lab report must follow the **"3CC3 Lab Policies and Report Writing.pdf"** document which is posted on avenue to learn. It should also include the following

1. What are the main components of a servomotor?
2. Explain the purpose and the working of the Mathscript Nodes in Exercise 2
3. Explain the Code in the MathScript Node of Exercise 3, and the purpose of each "if" statement. What does the operators "==" and "|" do in the code [Hint: you can check the Matlab online help].
4. Explain why the Local Variables were used in the code.
5. List three applications of servomotors.